

HariVishnuN-192524319 / Operating-System-CSA0406- ▾

Code   Issues   Pull requests   Actions   Projects   Wiki   Security and quality   Insights

Settings

Commit beba1ea

HariVishnuN-192524319 authored 52 minutes ago 

Verified

Create CO2-AT3

main

1 parent [94083c4](#) commit beba1ea

1 file changed 

+104

↑ Top

Filter files...

- ▼  CO Assessments
- CO2-AT3

Search within code

▼ CO Assessments/CO2-AT3

...

 @@ -0,0 +1,104 @@  
1 + Q1. Deadlock Debugging:  
2 +  
3 + Debugging Analysis:  
4 + The deadlock occurs because both processes acquire the shared resources in different  
  orders.  
5 + P1 locks R1 first and then requests R2.  
6 + P2 locks R2 first and then requests R1.  
7 +  
8 + If both processes execute simultaneously:

```
9   + P1 holds R1 and waits for R2.
10  + P2 holds R2 and waits for R1.
11  + This creates a circular wait, causing a deadlock.
12  +
13  + Optimized Code:
14  + Process P1
15  + lock(R1);
16  + lock(R2);
17  + Critical Section
18  + unlock(R2);
19  + unlock(R1);
20  +
21  + Process P2
22  + lock(R1);
23  + lock(R2);
24  + Critical Section
25  + unlock(R2);
26  + unlock(R1);
27  +
28  + Optimization:
29  + Maintain a fixed resource allocation order (R1 → R2) for all processes. This removes
    + the circular wait condition and prevents deadlock.
30  +
31  + Result:Deadlock is eliminated without changing the process logic.
32  +
33  + Q2. CPU Utilization Problem:
34  +
35  + Debugging Analysis
36  +
37  + The scheduler repeatedly shows:
38  + Ready → Waiting → Ready → Waiting
39  +
40  + This indicates that processes are I/O-bound. They frequently wait for I/O operations,
    + leaving the CPU idle even though processes exist in the system. As a result, CPU
    + utilization remains at only 30%.
41  +
42  + Optimization:
43  + Use asynchronous (non-blocking) I/O so processes do not block unnecessarily.
44  + Increase the degree of multiprogramming so another process can execute while one
    + waits.
```

```
45 + Use a scheduler such as Multilevel Feedback Queue (MLFQ) or optimize Round Robin
    parameters.
46 +
47 + Result:
48 + CPU idle time decreases and CPU utilization improves significantly.
49 +
50 + Q3. Memory Bottleneck Debugging:
51 +
52 + Debugging Analysis
53 +
54 + Memory usage:
55 +
56 + Code = 5 MB
57 + Heap = 150 MB
58 + Stack = 25 × 1 MB = 25 MB
59 + Total = 180 MB
60 +
61 + Although total memory is less than 2 GB, the application experiences frequent page
    faults because multiple threads access large memory regions, increasing memory
    pressure and reducing cache efficiency.
62 +
63 + Optimization:
64 +
65 + Optimization 1: Use a thread pool to reduce the number of active threads.
66 +
67 + Optimization 2: Optimize heap allocation by using memory pools, releasing unused
    memory, and improving data locality.
68 +
69 + Result:
70 + Page faults decrease, memory performance improves, and execution becomes faster.
71 +
72 + Q4. Starvation Debugging:
73 +
74 + Debugging Analysis:
75 +
76 + P5 belongs to Q3 (Batch Queue), which uses FCFS scheduling. Since higher-priority
    queues (Q1 and Q2) continuously receive new processes, P5 never gets CPU time.
77 + This is a classic case of starvation.
78 +
79 + Optimization:
```

```
80 + Implement Aging, where the priority of a process increases the longer it waits.
81 + Alternatively, use a Multilevel Feedback Queue (MLFQ) so long-waiting processes can
    move to higher-priority queues.
82 +
83 + Result:
84 + Every process eventually receives CPU time, ensuring fairness and preventing
    starvation.
85 +
86 + Q5. Context-Switch Overhead Debugging:
87 +
88 + Debugging Analysis:
89 +
90 + Given:
91 + Context switches = 20,000/sec
92 + Average burst time = 2 ms
93 + Time quantum = 1 ms
94 + High I/O frequency
95 +
96 + Since the time quantum (1 ms) is smaller than the average CPU burst (2 ms), most
    threads are preempted before completing. Frequent I/O waits also cause repeated
    scheduling, resulting in excessive context switching.
97 +
98 + Optimization:
99 + Optimization 1: Increase the time quantum to 4-5 ms so threads can complete more work
    before preemption.
100 +
101 + Optimization 2: Reduce I/O wait time using asynchronous I/O, buffering, or batching of
    I/O requests.
102 +
103 + Result:
104 + The number of context switches decreases, CPU overhead is reduced, and overall system
    performance improves.
```

**Comments** 0 Lock conversation

Comment

**Subscribe**

You're not receiving notifications from this thread.